



Sede Regional Brunca

Campus Pérez Zeledón

I Proyecto

II Ciclo (Grupo: 83)

Estudiante:

Anddy Zuñiga Vega

Profesor:

Pablo Andrés Venegas Elizondo

27 de Septiembre del 2026

1. INTRODUCCIÓN Y ARQUITECTURA

El proyecto "Tetris" consiste en el desarrollo de un juego interactivo utilizando C++ EN Zinjai y la biblioteca multimedia SFML 2.5. El objetivo principal fue la implementación manual de estructuras de datos dinámicas y la gestión directa de memoria mediante punteros.

A nivel de arquitectura, se emplea un patrón similar al de Modelo Vista Controlador (MVC). La lógica y las mecánicas matemáticas se encuentran en el modelo (Juego). Por su parte, las texturas y procesamiento de entradas del usuario se delegan al controlador gráfico (GestorUI). Esta separación permite que el motor matemático corra independientemente de la resolución gráfica o la interfaz.

2. MECÁNICAS CENTRALES

Las colisiones y las rotaciones se verifican evaluando el estado de ocupación de las celdas mediante la función asignada a la clase **Juego**. Para optimizar el rendimiento y evitar cálculos trigonométricos en tiempo real, las rotaciones de las figuras se modelaron utilizando matrices multidimensionales (4D). Además, se aplicó una regla estricta de evasión de "Wall-Kicks", si la rotación solicitada provoca una superposición simplemente niega el movimiento.

3. ANÁLISIS DE ESTRUCTURAS Y RESPUESTAS OBLIGATORIAS

3.1. Ordenamiento y Complejidad Asintótica

Pregunta 1: Comparación de $O(n^2)$ vs $O(n \log n)$ en el ranking.

La diferencia de rendimiento entre Insertion Sort ($O(n^2)$) y Quicksort ($O(n \log n)$) se vuelve empíricamente evidente a partir de un $N \approx 1000$ registros.

Para conjuntos pequeños ($N < 50$), ambos algoritmos resuelven en fracciones de milisegundos prácticamente igual, pero al escalar a 10,000 registros, el Quicksort optimizado (usando Mediana de Tres para evitar la degeneración de la partición de Lomuto) resuelve el ordenamiento de forma casi instantánea, mientras que el Insertion Sort provoca un retraso notable que congela el hilo de ejecución. Este comportamiento **coincide con la notación asintótica**, eso es que la curva de crecimiento cuadrática exige millones de operaciones (crecimiento parabólico), mientras que la logarítmica tipo lineal como que aplanar radicalmente el número de instrucciones que procesa la CPU.

Comparacion de algoritmos de ordenamiento
Insercion ($O(n^2)$) vs Quicksort ($O(n \log n)$)
(Tiempo promedio de 10 corridas por algoritmo, en milisegundos)

Tamano	Insercion ms	Quick ms	Mismo resultado de datos ordenados	Tiempo igual
10	0.000	0.000	SI	SI
100	0.000	0.000	SI	SI
1000	0.101	0.101	SI	NO
10000	3.248	3.248	SI	NO

Para 10000 datos: insercion 0.0839325 s, quicksort 0.003248 s
(Resultados guardados en ordenamientos.txt)

Se puede notar que el resultado será el mismo, pero el tiempo de ordenamiento si es donde se encuentra una diferencia.

3.2. Modelado del Tablero y Limpieza de Líneas

Pregunta 2: Representación del tablero como lista enlazada de filas.

Representar el tablero de Tetris como una lista enlazada simple o doble de filas es una forma teóricamente superior de modelar la limpieza respecto a una matriz estática.

En una matriz 2D, eliminar una fila completada en la base exige recorrer todas las filas superiores y copiar sus valores una celda abajo, lo que conlleva una complejidad.

Pero, si el tablero es una lista de punteros a arreglos unidimensionales (filas), **el costo de eliminar una fila completa y reinsertar una fila vacía en la cima es estrictamente de $O(1)$** . Solo se requiere desenlazar el nodo de la fila destruida, enlazar los punteros de la fila superior con la inferior, reciclar ese nodo limpiando sus valores, e insertarlo como nueva cabeza (frente) de la lista. Eso sí, son muchas reasignaciones de punteros, independientemente del ancho del tablero.

3.3. Sistema de Replay

Pregunta 3: Necesidad de la Lista Doblemente Enlazada y sus costos.

El sistema de Replay almacena instantáneas completas del juego (Estado) para navegar en el tiempo. Requiere obligatoriamente una **lista doblemente enlazada** porque se necesita desplazar bidireccionalmente, es decir tanto hacia el pasado para revisar un error, como hacia el futuro para reanudar la vista. Una lista simplemente enlazada solo guarda referencias al nodo siguiente, por lo que retroceder implicaría volver a iterar desde el inicio (frente) de la lista en $O(N)$ operaciones cada vez que se pulsa la tecla izquierda. En la implementación que se hizo en el código el costo de avanzar (rehacer()) o retroceder (deshacer()) un paso temporal es de $O(1)$. Esto se logra desplazando un único puntero interno (actual = actual->siguiente o actual = actual->anterior).

3.4. Gestión Cronológica de Eventos

Pregunta 4: Garantía de la Cola de Eventos y su complejidad.

La ColaEventos hace que el evento más próximo a activarse siempre esté al frente al momento de realizar la inserción, evadiendo la necesidad de invocar algoritmos de ordenamiento costosos de forma repetitiva.

La implementación de la función insertar(const Evento& e) recorre secuencialmente la lista enlazada comparando el atributo momento de disparo. Se detiene en el instante exacto donde encuentra un nodo cuyo momento es mayor al nuevo evento, ajustando los punteros en esa posición intermedia para no alterar el orden que estaba antes. Debido a este recorrido secuencial de búsqueda, **la complejidad de insertar un nuevo evento es de $O(N)$** en el peor de los casos (cuando el evento debe ir al final de la cola). La ventaja es que extraer el evento por el frente (pop()) retiene una eficiencia de $O(1)$.

4. CONCLUSIÓN

El desarrollo de este proyecto permitió comprender conceptos sobre la gestión dinámica de memoria y la importancia de la complejidad algorítmica en el rendimiento real. El uso de punteros hizo que se implementara una limpieza en los destructores para evitar fallos de segmentación o memory leak, por último, una simple decisión en las estructuras de datos (como el uso de apuntadores dobles para desplazamientos temporales $O(1)$ o la elección del Quicksort) define la usabilidad de un programa interactivo, valida de forma práctica la teoría estudiada durante el curso.